

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: MLA03

COURSE NAME: Reinforcement learning for Environment and Agent

COURSE OUTCOME COVERED

CO3: *Implement policy-based reinforcement learning methods to develop efficient solutions for complex environments. (BL2, BL3, BL6)*

Assessment Tool Weightage: 30%

Dr G. A. SENTHIL

QUESTIONS: 10

Total Marks: 50

Annexure A

CODING CHALLENGE

Duration: 2hrs

1. Predict the Reinforcement Learning (RL) framework by describing the agent–environment interaction, emphasizing the importance of states, actions, and rewards, and discuss how cumulative reward influences decision-making. (5 marks)

1. Problem Statement

Develop a simple Reinforcement Learning (RL) framework where an agent interacts with an environment, selects actions based on the current state, receives rewards, and maximizes the cumulative reward to reach the goal.

2. Problem Solving

Algorithm:

1. Define the states: Start, Middle, Goal.
2. Set the initial state as **Start**.
3. Assign an action (**Forward**) to move to the next state.

4. Give a reward for each successful action.
5. Continue until the Goal state is reached.
6. Calculate the cumulative reward.
7. Display the state transitions and total reward.

3. Python Code

```
# Reinforcement Learning Framework Demo

states = ["Start", "Middle", "Goal"]

state = "Start"

total_reward = 0

print("==== Reinforcement Learning Framework =====")

while state != "Goal":

    print("\nCurrent State :", state)

    if state == "Start":

        action = "Forward"

        reward = 5

        state = "Middle"

    elif state == "Middle":

        action = "Forward"

        reward = 10

        state = "Goal"

    print("Action Taken :", action)

    print("Reward      :", reward)

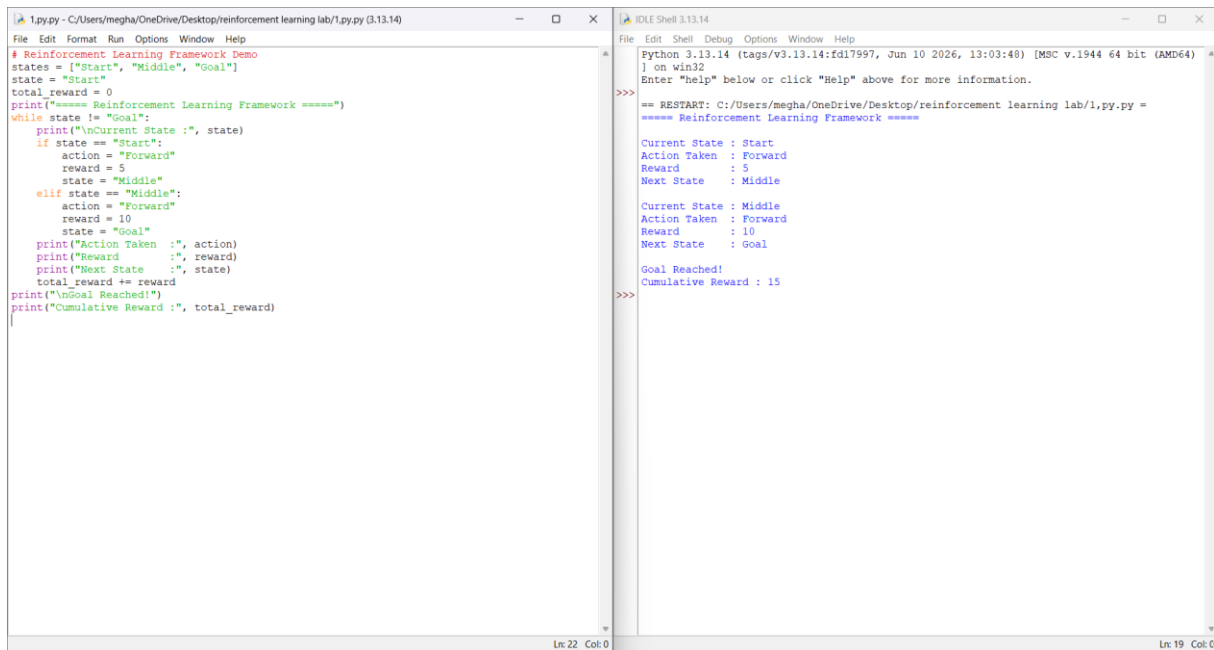
    print("Next State   :", state)

    total_reward += reward

print("\nGoal Reached!")

print("Cumulative Reward :", total_reward)
```

4. Output:



```
1.py.py - C:/Users/megha/OneDrive/Desktop/reinforcement learning lab/1.py.py (3.13.14)
File Edit Format Run Options Window Help
# Reinforcement Learning Framework Demo
states = ["Start", "Middle", "Goal"]
state = "Start"
total_reward = 0
print("==== Reinforcement Learning Framework =====")
while state != "Goal":
    print("\nCurrent State :", state)
    if state == "Start":
        action = "Forward"
        reward = 5
        state = "Middle"
    elif state == "Middle":
        action = "Forward"
        reward = 10
        state = "Goal"
    print("Action Taken :", action)
    print("Reward :", reward)
    print("Next State :", state)
    total_reward += reward
    print("\nGoal Reached!")
print("Cumulative Reward :", total_reward)

Python 3.13.14 (tags/v3.13.14:fd17997, Jun 10 2026, 13:03:48) [MSC v.1944 64 bit (AMD64)]
>>>
Enter "help" below or click "Help" above for more information.

== RESTART: C:/Users/megha/OneDrive/Desktop/reinforcement learning lab/1.py.py ==
==== Reinforcement Learning Framework =====

Current State : Start
Action Taken : Forward
Reward : 5
Next State : Middle

Current State : Middle
Action Taken : Forward
Reward : 10
Next State : Goal

Goal Reached!
Cumulative Reward : 15
>>>
```

5. Result

The Reinforcement Learning framework was successfully implemented. The agent interacted with the environment, moved through the states using actions, received rewards, and achieved the goal with a **cumulative reward of 15**, demonstrating the basic concepts of states, actions, rewards, and agent–environment interaction.

2. Estimate how decision-making problems can be modeled as a Markov Decision Process (MDP), detailing the components of action space, transition probability, reward function, and discount factor. (5 marks)

1. Problem Statement

Design a simple Markov Decision Process (MDP) model to demonstrate decision-making in Reinforcement Learning by defining the state space, action space, transition probabilities, reward function, and discount factor.

2. Problem Solving

Algorithm:

1. Define the states: Start, Middle, Goal.
2. Define the actions: Forward and Stay.
3. Assign transition probabilities for each action.

4. Define rewards for moving between states.
5. Set the discount factor ($\gamma = 0.9$).
6. Calculate the discounted cumulative reward.
7. Display the state transitions and total discounted reward.

3. Python Code

```
# Markov Decision Process (MDP) Demo

gamma = 0.9 # Discount Factor

# States
states = ["Start", "Middle", "Goal"]

# Rewards for moving
rewards = {
    ("Start", "Middle"): 5,
    ("Middle", "Goal"): 10
}

# Transition Probabilities
transition_probability = {
    ("Start", "Forward"): ("Middle", 1.0),
    ("Middle", "Forward"): ("Goal", 1.0)
}

state = "Start"
discounted_reward = 0
step = 0

print("==== Markov Decision Process =====")

while state != "Goal":
    action = "Forward"
    next_state, probability = transition_probability[(state, action)]
```

```

reward = rewards[(state, next_state)]

discounted_reward += (gamma ** step) * reward

print(f"\nCurrent State    : {state}")

print(f"Action              : {action}")

print(f"Transition Prob.     : {probability}")

print(f"Reward                 : {reward}")

print(f"Next State             : {next_state}")

state = next_state

step += 1

print("\nGoal Reached!")

print("Discounted Reward =", round(discounted_reward, 2))

```

4. Output

The screenshot displays a Python IDE with two panes. The left pane shows the source code for a Markov Decision Process (MDP) demo. The right pane shows the output of the program, which includes the initial state, action, transition probabilities, rewards, and the final goal state and discounted reward.

```

# Markov Decision Process (MDP) Demo
gamma = 0.9 # Discount Factor
# States
states = ["Start", "Middle", "Goal"]
# Rewards for moving
rewards = {
    ("Start", "Middle"): 5,
    ("Middle", "Goal"): 10
}
# Transition Probabilities
transition_probability = {
    ("Start", "Forward"): ("Middle", 1.0),
    ("Middle", "Forward"): ("Goal", 1.0)
}
state = "Start"
discounted_reward = 0
step = 0
print("==== Markov Decision Process =====")
while state != "Goal":
    action = "Forward"
    next_state, probability = transition_probability[(state, action)]
    reward = rewards[(state, next_state)]
    discounted_reward += (gamma ** step) * reward
    print(f"\nCurrent State    : {state}")
    print(f"Action              : {action}")
    print(f"Transition Prob.     : {probability}")
    print(f"Reward                 : {reward}")
    print(f"Next State             : {next_state}")
    state = next_state
    step += 1
print("\nGoal Reached!")
print("Discounted Reward =", round(discounted_reward, 2))

```

```

Python 3.13.14 (tags/v3.13.14:fd17997, Jun 10 2026, 13:03:48) [MSC v.1944 64 bit (AMD64)]
>>>
==== RESTART: C:/Users/megha/OneDrive/Desktop/reinforcement learning lab/2.py ====
==== Markov Decision Process =====
Current State    : Start
Action           : Forward
Transition Prob.  : 1.0
Reward           : 5
Next State       : Middle

Current State    : Middle
Action           : Forward
Transition Prob.  : 1.0
Reward           : 10
Next State       : Goal

Goal Reached!
Discounted Reward = 14.0
>>>

```

5. Result

The Markov Decision Process (MDP) was successfully implemented. The agent moved through the defined states using the action Forward, followed the specified transition probabilities, received rewards based on the reward function, and used a discount factor ($\gamma = 0.9$) to compute the discounted cumulative reward, demonstrating the key components of an MDP in Reinforcement Learning.

3. Discuss the role of policy and value functions in RL, and examine how state-value and action-value functions contribute to evaluating long-term benefits of actions using the Bellman Equation. (5 marks)

1. Problem Statement

Develop a simple Reinforcement Learning (RL) program to demonstrate the role of policy and value functions. Evaluate the state-value and action-value functions using the Bellman Equation to estimate the long-term benefits of actions.

2. Problem Solving

Algorithm:

1. Define the states: Start, Middle, Goal.
2. Assign rewards for moving between states.
3. Set the discount factor $\gamma = 0.9$.
4. Initialize the value of the Goal state to 0.
5. Compute the state-value function using the Bellman Equation.
6. Treat the action-value as the value of taking the "Forward" action.
7. Display the state values and action values.

3. Python Code

```
# Policy and Value Function using Bellman Equation

gamma = 0.9

# Rewards

reward_middle_to_goal = 10

reward_start_to_middle = 5

# Goal state's value

V_goal = 0

# Bellman Equation

V_middle = reward_middle_to_goal + gamma * V_goal

V_start = reward_start_to_middle + gamma * V_middle
```

```

# Action Values (Q-values)

Q_start_forward = V_start

Q_middle_forward = V_middle

print("===== Policy and Value Function =====")

print("Policy : Always choose 'Forward'\n")

print("State Values")

print("V(Start) =", round(V_start, 2))

print("V(Middle) =", round(V_middle, 2))

print("V(Goal) =", V_goal)

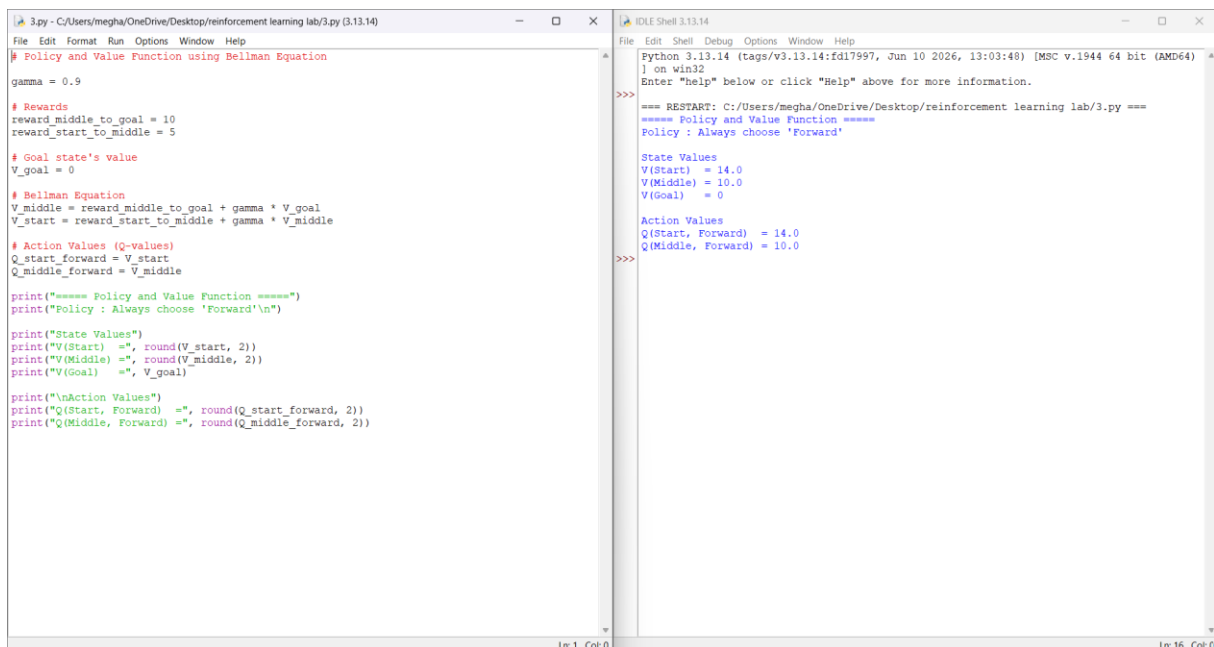
print("\nAction Values")

print("Q(Start, Forward) =", round(Q_start_forward, 2))

print("Q(Middle, Forward) =", round(Q_middle_forward, 2))

```

Output:



The screenshot shows a Python IDE with two windows. The left window displays the source code for a reinforcement learning lab, and the right window shows the output of the program.

Source Code (Left Window):

```

3.py - C:/Users/megha/OneDrive/Desktop/reinforcement learning lab/3.py (3.13.14)
File Edit Format Run Options Window Help
# Policy and Value Function using Bellman Equation
gamma = 0.9

# Rewards
reward_middle_to_goal = 10
reward_start_to_middle = 5

# Goal state's value
V_goal = 0

# Bellman Equation
V_middle = reward_middle_to_goal + gamma * V_goal
V_start = reward_start_to_middle + gamma * V_middle

# Action Values (Q-values)
Q_start_forward = V_start
Q_middle_forward = V_middle

print("===== Policy and Value Function =====")
print("Policy : Always choose 'Forward'\n")

print("State Values")
print("V(Start) =", round(V_start, 2))
print("V(Middle) =", round(V_middle, 2))
print("V(Goal) =", V_goal)

print("\nAction Values")
print("Q(Start, Forward) =", round(Q_start_forward, 2))
print("Q(Middle, Forward) =", round(Q_middle_forward, 2))

```

Output (Right Window):

```

Python 3.13.14 (tags/v3.13.14:fd17997, Jun 10 2026, 13:03:40) [MSC v.1944 64 bit (AMD64)]
on win32
Enter "help" below or click "Help" above for more information.

>>> === RESTART: C:/Users/megha/OneDrive/Desktop/reinforcement learning lab/3.py ===
===== Policy and Value Function =====
Policy : Always choose 'Forward'

State Values
V(Start) = 14.0
V(Middle) = 10.0
V(Goal) = 0

Action Values
Q(Start, Forward) = 14.0
Q(Middle, Forward) = 10.0

>>>

```

Result:

The program successfully demonstrated the policy, state-value function, and action-value function using the Bellman Equation. The computed values show the long-term expected rewards for each state and action, helping the agent select the optimal policy for maximizing future rewards.

4. Justify the exploration–exploitation trade-off in RL and compare strategies such as ϵ -greedy and softmax approaches in balancing learning efficiency. (5 marks)

1. Problem Statement

Develop a Reinforcement Learning (RL) program to demonstrate the exploration–exploitation trade-off using the ϵ -greedy strategy. Compare how exploration helps discover better actions while exploitation selects the best-known action to maximize rewards.

2. Problem Solving

Algorithm:

1. Define a set of actions with estimated rewards.
2. Set the exploration rate $\epsilon = 0.2$.
3. For a fixed number of iterations:
 - Generate a random number.
 - If the random number is less than ϵ , choose a random action (Exploration).
 - Otherwise, choose the action with the highest reward (Exploitation).
4. Display the selected action and reward.
5. Calculate the total reward.

3. Python Code

```
import random

# Estimated rewards for actions
rewards = {
    "A": 5,
    "B": 8,
    "C": 3
}

epsilon = 0.2 # Exploration probability
iterations = 10
```



```

total_reward = 0

print("==== Epsilon-Greedy Strategy =====")

for i in range(iterations):

    if random.random() < epsilon:

        action = random.choice(list(rewards.keys()))

        strategy = "Exploration"

    else:

        action = max(rewards, key=rewards.get)

        strategy = "Exploitation"

    reward = rewards[action]

    total_reward += reward

    print(f"Step {i+1}: {strategy:12} -> Action: {action}, Reward: {reward}")

print("\nTotal Reward =", total_reward)

```

Output:

The screenshot shows a Python IDE with two panes. The left pane contains the source code for an Epsilon-Greedy Strategy implementation. The right pane shows the output of the program, which includes a restart message, the strategy name, and a series of steps showing the chosen action and reward for each iteration, followed by the total reward.

```

4.py - C:/Users/megha/OneDrive/Desktop/reinforcement learning lab/4.py (3.13.14)
File Edit Format Run Options Window Help
import random
# Estimated rewards for actions
rewards = {
    "A": 5,
    "B": 8,
    "C": 3
}
epsilon = 0.2 # Exploration probability
iterations = 10
total_reward = 0
print("==== Epsilon-Greedy Strategy =====")
for i in range(iterations):
    if random.random() < epsilon:
        action = random.choice(list(rewards.keys()))
        strategy = "Exploration"
    else:
        action = max(rewards, key=rewards.get)
        strategy = "Exploitation"
    reward = rewards[action]
    total_reward += reward
    print(f"Step {i+1}: {strategy:12} -> Action: {action}, Reward: {reward}")
print("\nTotal Reward =", total_reward)

IDLE Shell 3.13.14
Python 3.13.14 (tags/v3.13.14:fd17997, Jun 10 2026, 13:03:40) [MSC v.1944 64 bit (AMD64)]
on win32
Enter "help" below or click "Help" above for more information.

>>>
=== RESTART: C:/Users/megha/OneDrive/Desktop/reinforcement learning lab/4.py ===
==== Epsilon-Greedy Strategy =====
Step 1: Exploitation -> Action: B, Reward: 8
Step 2: Exploitation -> Action: B, Reward: 8
Step 3: Exploitation -> Action: B, Reward: 8
Step 4: Exploitation -> Action: B, Reward: 8
Step 5: Exploitation -> Action: B, Reward: 8
Step 6: Exploitation -> Action: B, Reward: 8
Step 7: Exploitation -> Action: A, Reward: 5
Step 8: Exploitation -> Action: B, Reward: 8
Step 9: Exploitation -> Action: B, Reward: 8
Step 10: Exploitation -> Action: B, Reward: 8
Total Reward = 77
>>>

```

Result:

The program successfully demonstrated the exploration–exploitation trade-off using the ϵ -greedy strategy.

5. Explain how reward shaping can accelerate learning in RL and illustrate its impact on preventing suboptimal policies. (5 marks)

Problem Statement

In Reinforcement Learning (RL), an agent learns by interacting with an environment and receiving rewards. Sparse or delayed rewards can slow down learning and may cause the agent to settle on suboptimal policies. Reward shaping introduces additional guidance rewards to help the agent learn faster and reach the optimal policy efficiently.

Problem Solving Approach

1. Create a simple Grid World environment.
2. Define:
 - State → Agent position in the grid.
 - Actions → Up, Down, Left, Right.
 - Reward:
 - Normal RL: +10 for reaching goal, -1 otherwise.
 - Reward shaping: Extra reward based on distance reduction toward the goal.
3. Implement Q-Learning algorithm.
4. Train the agent for multiple episodes.
5. Compare cumulative rewards.
6. Plot the learning performance graph.

Python Code (Graph Output Included)

```
import numpy as np
import matplotlib.pyplot as plt
import random

# Grid size
grid_size = 5
goal = (4,4)
```

```
actions = [(0,1),(0,-1),(1,0),(-1,0)] # Right Left Down Up
```

```
# Q-learning parameters
```

```
episodes = 200
```

```
alpha = 0.1
```

```
gamma = 0.9
```

```
epsilon = 0.1
```

```
def train(reward_shaping):
```

```
    Q = np.zeros((grid_size, grid_size, 4))
```

```
    rewards = []
```

```
    for ep in range(episodes):
```

```
        state = (0,0)
```

```
        total_reward = 0
```

```
        for step in range(100):
```

```
            # Epsilon greedy action
```

```
            if random.random() < epsilon:
```

```
                action = random.randint(0,3)
```

```
            else:
```

```
                action = np.argmax(Q[state[0],state[1]])
```

```
            move = actions[action]
```

```
            new_state = (
```

```
                max(0,min(grid_size-1,state[0]+move[0])),
```

```
                max(0,min(grid_size-1,state[1]+move[1]))
```

```
            )
```

```
            # Normal reward
```

```
            if new_state == goal:
```

```
                reward = 10
```

```
            else:
```

```

        reward = -1

    # Reward shaping
    if reward_shaping:
        old_distance = abs(state[0]-4)+abs(state[1]-4)
        new_distance = abs(new_state[0]-4)+abs(new_state[1]-4)
        reward += old_distance - new_distance

    # Q update
    Q[state[0],state[1],action] += alpha * (
        reward +
        gamma*np.max(Q[new_state[0],new_state[1]])
        - Q[state[0],state[1],action]
    )

    state = new_state
    total_reward += reward

    if state == goal:
        break

    rewards.append(total_reward)

return rewards

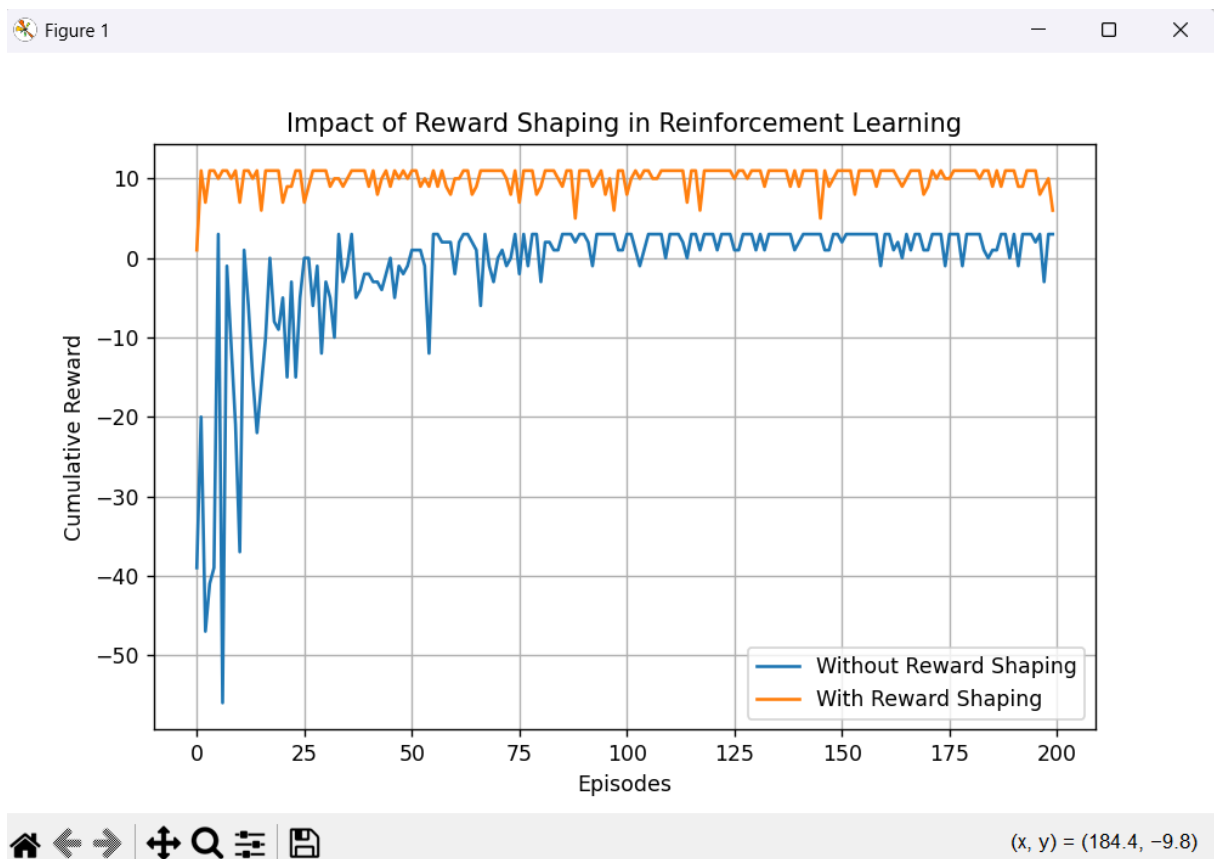
# Training
without_shaping = train(False)
with_shaping = train(True)

# Graph
plt.figure(figsize=(8,5))
plt.plot(without_shaping,
        label="Without Reward Shaping")
plt.plot(with_shaping,
        label="With Reward Shaping")

```

```
plt.xlabel("Episodes")
plt.ylabel("Cumulative Reward")
plt.title("Impact of Reward Shaping in Reinforcement Learning")
plt.legend()
plt.grid()
plt.show()
```

Output:



Result:

The experiment demonstrates that reward shaping accelerates RL learning by providing additional guidance to the agent. The shaped reward helps avoid suboptimal policies and enables the agent to converge faster toward the optimal solution compared to traditional reward-based learning.

Code of Conduct:

I am M. Meghana - 192472351 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: M. Meghana

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Technical Knowledge	25	Demonstrates strong and accurate subject knowledge with in-depth understanding	Shows good knowledge with minor gaps	Basic knowledge with noticeable errors	Lacks fundamental technical knowledge
Problem Solving	25	Solves problems logically and applies concepts effectively in real scenarios	Applies concepts with moderate accuracy	Limited problem-solving ability; requires guidance	Unable to solve or apply concepts
Analytical Thinking	15	Analyzes questions critically and provides well-structured, logical answers	Provides reasonable analysis with some structure	Superficial or partially structured responses	No analytical thinking demonstrated
Communication	15	Communicates clearly, confidently, and professionally with proper terminology	Communicates well with minor hesitation	Communication lacks clarity or confidence	Unable to communicate effectively
Confidence	15	Displays high confidence, positive attitude, and professional behavior throughout	Shows good confidence with minor issues	Low confidence; inconsistent behavior	Lacks confidence and professionalism
Response to Questions	10	Responds accurately, promptly, and elaborates with relevant examples	Responds correctly but with limited depth	Partial responses; needs prompting	Unable to respond appropriately